

Lab2 Lexical Analysis

lab2 ddl: 2026/4/8 23:59

1. Introduction

Why a compiler needs lexical scanner?

The lexical scanner plays a crucial role in the initial stages of compilation by transforming the source code into a stream of tokens, which are then used by the parser for further analysis and translation into machine code.

What is Flexc++?

Flexc++ is a tool for generating lexical scanners: programs recognizing patterns in text. It reads one or more files from input, containing rules: regular expressions, optionally associated with C++ code. From this **Flexc++** generates several files, containing the declaration and implementation of a class (Scanner by default). The member function `lex` is used to analyze input: it looks for text matching the regular expressions. Whenever it finds a match, it executes the associated C++ code.

2. Lab Description

In lab2, you'll use **flexc++** to implement a lexical scanner for the Tiger language. *Tiger Language Reference Manual* (Appendix A) describes, among other things, the lexical tokens of Tiger.

Your lexical scanner for the Tiger language should include the following parts:

- Basic specifications;
- Comments handling;
- Strings handling;
- Error handling;

You will code under the directory `src/tiger/lex` and modify two files `tiger.lex` and `scanner.h`. We have already given some hints for you in the comments. Other related files for this lab are `src/tiger/errormsg/errormsg.*` and `src/tiger/main/test_lex.cc`. `errormsg.*` files implement the error message module, which is useful for producing error messages with file names and line numbers.

`test_lex.cc` helps you build the executable file for lab2.

The testcases for this lab are in `testdata/lab2/testcases/`, and the correct outputs of the testcases can be found in `testdata/lab2/refs/`.

Hints:

- You will need to code in `tiger.lex` and maybe in `scanner.h`. The file `scanner.h` is the class declaration of your scanner, which is initially generated by flexc++ when they do not exist for the first time. Flexc++ will also generate `lex.cc` and `scannerbase.h`, which implement your scanner class according to your rules defined in `tiger.lex`.
- The generated scanner class has many member functions that you can directly call in the rules of `tiger.lex`. We have also provided some helper functions (`adjust` and `adjustStr`) in `scanner.h`. You can define your own variables and functions inside `scanner.h`.
- When reading the *Tiger Language Reference Manual* (Appendix A), pay particular attention to the paragraphs with the headings **Identifiers**, **Comments**, **Integer literal**, and **String literal**.
- Note that you don't need to set the semantic value of each token like the codes(`yylval`) in the textbook, because we have already set it for you in `parser.ih`. What you need to do is to return each token's enumeration value.
- For string tokens, you are required to convert the string literal to the real string value and set your scanner's current matched to it using `setMatched()` member function. For example, a matched string literal maybe `"hello\n"`, you should convert the escape `"\n"` to the real linebreak character. The `/*comments*/` in the testcases should be ignored by the lexer you implemented.
- Scanner member functions that you may use:
 - `matched()`
 - `setMatched()`
 - `begin(StartCondition_::COMMENT)` **Notice:** The enum type `StartCondition_` has **one** underlines.Refer to [Start Conditions](#) for comment handling and string handling.

Notice: Before you start this lab, you should carefully read chapter 2 of the textbook, and you may need to get some reference from the [flexc++ manual](#) and the *Tiger Language Reference Manual* (Appendix A). As usual, please don't hesitate to reach out to the TA if you have any questions about this lab!

3. Enviroment

We've made an update to the docker image (zip and sudo commands can now be used), so please use the latest image from DockerHub. The pull process will be done automatically.

Code block

```
1 docker run -it --privileged -p 2121:22 -v ${PWD}:/home/stu/tiger-compiler
  lyf726/tiger-compiler-env:latest
```

You will use the same code framework that you had set up when you worked on lab1. What you need to do now is to pull the latest update of the code framework if there are any. You may have to do some code merging jobs.

Code block

```
1 shell% git fetch upstream
2 shell% git checkout -b lab2 upstream/lab2
3 shell% git push -u origin
4 shell% git merge lab1 (some students may finish lab1 on master, thus merge
  master instead)
```

4. Grade Test

The lab environment contains a grading script named **scripts/grade.sh**, you can use it to evaluate your code, and that's how we grade your code, too. If you pass all the tests, the script will print a successful hint, otherwise, it will output some error messages. You can execute the script with the following commands.

Code block

```
1 shell% make gradelab2 or bash ./scripts/grade.sh lab2
2 shell% ...
3 shell% [^_^]: Pass** _#If you pass all the tests, you will see these messages.
4 shell% SCORE: 100
```

5. Submission Instructions

1. Compress to one directory, using `make ziplab2` command.
2. Hand in this directory to gradescope website. (you can see previous documents for more information)
3. Report is required for lab2, please refer to the following report template and submit the report to GradeScope in **pdf** version.

# lab2 (Mini Report)	MAR 17, 2026 12:00 PM APR 8, 2026 11:59 PM Late Due Date: APR 15, 2026 11:59 PM	0	0%	☐	ON	⋮
#Lab2 Lexical Analysis	MAR 17, 2026 11:59 AM APR 8, 2026 11:59 PM Late Due Date: APR 15, 2026 11:59 PM	1	100%	☐	ON	⋮

#Lab2 mini-report

Code Handling Practices and Lexer Features

- How you handle comments
- How you handle strings
- Error handling
- End-of-file handling
- Other interesting features of your lexer

Challenges and Bug Fixes

- Challenges during your implementation
- Bug fix log

⚠ Write your report following the format outlined above, filling in each subsection accordingly. Ensure your report is concise, devoid of superfluous content, and presented in a clean, clear, and organized manner. 😊

! **Remember to submit before the deadline to avoid late penalties!**

✍ BTW, the language you use to finish the report will not make a difference in scoring.

6. Key Takeaways

After completing Lab 2, you will have gained experience in the following areas:

- **Using Flexc++:** You will have gained practical experience using Flexc++ to define lexical rules and implement a scanner for a specific language (Tiger in this case). This includes understanding regular expressions and how to associate them with C++ code for tokenization.
- **Lexical Analysis:** You will have a deeper understanding of the role of a lexical scanner in the compilation process, specifically how it transforms source code into a stream of tokens for further processing.

- **Handling Different Token Types:** You will have gained experience implementing rules for handling different types of tokens, including identifiers, comments, integer literals, and string literals. This includes handling special cases like escape sequences in strings.
- **Error Handling:** You will have practiced implementing basic error handling mechanisms within the scanner to identify and report potential issues in the source code.
- **Understanding Tiger Language:** You will have gained deeper familiarity with the specific syntax and lexical structure of the Tiger language.



So, dive in, explore, and enjoy the learning process!